✅ Q1

```
arr = np.array([[1,2,3],
                [4,5,6]])

for x in np.nditer(arr):
    print(x, end=" ")
```

🔹 Output

```
1 2 3 4 5 6
```

🔹 Why?

Default is **C-order (row-wise)**.

✅ Q2

```
arr = np.array([1,2,3,4])

for x in np.nditer(arr, op_flags=['readwrite']):
    x[...] = x * 3

print(arr)
```

🔹 Output

```
[ 3  6  9 12]
```

🔹 Why?

`readwrite` allows modification.
`x[...]` modifies original memory.

✅ Q3 (Trap)

```
arr = np.array([1,2,3,4])

for x in np.nditer(arr, op_flags=['readwrite']):
    x = x * 3

print(arr)
```

🔹 Output

```
[1 2 3 4]
```

🔹 Why?

`x = x * 3` only reassigns the local variable.
It does NOT modify the array memory.

✅ Q4 (Transposed Order)

```
arr = np.arange(1,7).reshape(2,3).T

for x in np.nditer(arr):
    print(x, end=" ")
```

Transposed array:

```
[[1 4]
 [2 5]
 [3 6]]
```

🔹 Output

```
1 2 3 4 5 6
```

🔹 Why?

`nditer` still follows **memory layout (C-order of original)**,
not visual layout.

Transpose changes strides, not memory order.

✅ Q5 (Force Fortran Order)

```
arr = np.arange(1,7).reshape(2,3)

for x in np.nditer(arr, order='F'):
    print(x, end=" ")
```

Array:

```
[[1 2 3]
 [4 5 6]]
```

🔹 Output

```
1 4 2 5 3 6
```

Column-wise traversal.

✅ Q6 (Multiple Arrays)

```
a = np.array([1,2,3])
b = np.array([10,20,30])

for x, y in np.nditer([a,b]):
    print(x + y)
```

🔹 Output

```
11
22
33
```

Element-wise pairing.

✅ Q7 (Broadcasting)

```
a = np.array([1,2,3])
b = np.array([[10],
              [20],
              [30]])

for x, y in np.nditer([a, b]):
    print(x, y)
```

🔹 Iterations?

9 iterations.

Because:

- `a` shape → (3,)
- `b` shape → (3,1)
- Broadcast result → (3,3)

So 3×3 = 9 elements.

✅ Q8 (Write to One Operand)

```
a = np.array([1,2,3])
b = np.array([10,20,30])

for x, y in np.nditer([a,b], op_flags=[['readwrite'], ['readonly']]):
    x[...] = x + y

print(a)
print(b)
```

🔹 Output

```
[11 22 33]
[10 20 30]
```

Only `a` changes because:

- `a` → readwrite
- `b` → readonly

✅ Q9 (Transposed + Write)

```
arr = np.arange(6).reshape(2,3).T

for x in np.nditer(arr, op_flags=['readwrite']):
    x[...] = 0

print(arr)
```

🔹 Does it work?

Yes.

🔹 Output

```
[[0 0]
 [0 0]
 [0 0]]
```

🔹 Why?

Transpose is a **view**.
Modifying it modifies original memory.

✅ Q10 (Most Tricky)

```
arr = np.array([1,2,3,4])

it = np.nditer(arr, op_flags=['readwrite'])

for x in it:
    arr[0] = 999
    x[...] = x * 2

print(arr)
```

🔹 Step-by-step:

Iteration order: index 0 → 1 → 2 → 3

Iteration 1:

- arr[0] = 999
- x (points to arr[0]) → 999 * 2 = 1998

Array: `[1998, 2, 3, 4]`

Iteration 2:

- arr[0] = 999
- x (points to arr[1]) → 2 * 2 = 4

Array: `[999, 4, 3, 4]`

Iteration 3:

- arr[0] = 999
- x (points to arr[2]) → 3 * 2 = 6

Array: `[999, 4, 6, 4]`

Iteration 4:

- arr[0] = 999
- x (points to arr[3]) → 4 * 2 = 8

Final:

```
[999 4 6 8]
```

🎯 Final Summary

Question	Concept Tested
Q1	Default C-order
Q2	Proper modification
Q3	Local reassignment trap
Q4	Memory vs visual order
Q5	Fortran order
Q6	Multi-array iteration
Q7	Broadcasting
Q8	Operand flags
Q9	View behavior
Q10	Mutation during iteration